ASSIGNMENT 5: PRACTICAL REPORT

Taxonomy Evaluator with Docker and GitHub Actions

Nursultan Serikov, CSE-2501M, Astana IT University

3 November 2026

1. Objective, repository, and environment

The objective was to containerize the existing Python research CLI and automate its verification and delivery. GitHub Actions is the selected CI platform, following the task instruction. The evaluator's scientific implementation, CLI, five regression methods, and synthetic examples are preserved from Assignment 3. No new scientific dataset or measured dissertation accuracy is claimed.

Repository: github.com/thgank/taxonomy-evaluator.
Branch: `assignment5-github-actions`. The pipeline is `.github/workflows/ci.yml`; the container definition is `Dockerfile`. The tested code revision is `11d715859e23bd67b5515139f5306586e6de8302`.

Local verification used macOS 26.2 arm64 and Python 3.13.3. Actual container verification used Ubuntu 26.04.1 LTS x86_64 on GitHub, Docker Engine 29.4.2, and Python 3.13.18 inside the image. The host test matrix selected Python 3.11, 3.13, and 3.14. Exact runtime details are retained in the execution evidence, rather than inferred from the matrix names.

2. Implemented steps and automatic trigger

First, the baseline revision `cdea93a4` passed all five local regression methods and reproduced both synthetic benchmark settings. A separate task branch then added a minimal Dockerfile, restrictive Docker context, container check script, and README instructions. The base image is pinned by release and digest; the evaluator still requires no third-party runtime packages.

The workflow executes a nonempty test suite on each matrix interpreter. Its dependent delivery job records the revision and environment, builds an image tagged with that SHA, verifies offline container behavior, and delivers source, image, and JSON artifacts. Failed checks block delivery; diagnostic evidence is uploaded even on failure. Job timeouts are 10 minutes for tests and 15 minutes for delivery.

The implementation push produced successful run #3. A separate README commit triggered successful run #4, confirming automatic detection of later changes. A packaging correction then produced the final run #5 (41595705699). Its event is `push`, on the branch and SHA above, created at 07:20:50 UTC and completed by 07:21:18 UTC on 3 November 2026. No manual dispatch was used.

3. Measured results and evidence

All three test jobs passed five methods each: 15 method executions, not 15 distinct tests. The Docker build and six container invocation scenarios passed. Their expected exit codes were 0 for the baseline, filtered, and repeated evaluations; 1 for the failing quality gate; and 2 for an invalid threshold and attempted input overwrite.

Synthetic case	TP	FP	FN	Precision	Recall	F1
All edges	5	1	1	.8333	.8333	.8333
Threshold 0.5	5	0	1	1.0000	.8333	.9091

The check script asserts counts and metrics against known values, verifies input SHA-256 hashes, and compares two JSON reports byte for byte within the same image. Input mounts are read-only, results are written through a separate volume, and the container runs with networking disabled and a read-only root filesystem. The failed minimum-F1 test demonstrates rejection without deliberately breaking the repository.

The inspected image ID begins `sha256:32d752c800363c92`; the full ID, architecture, and revision label are in `container/image.json`. The image archive and runnable ZIP were downloaded, and both matched the workflow's SHA-256 checksum file. All run-specific evidence is under `assignment5/evidence/run-41595705699/`, including the complete delivery log, individual test logs, run/job metadata, build output, and evaluation JSON.

Screenshots `01-actions-run-5.png` and `02-delivery-stages-run-5.png` show the genuine GitHub run summary and delivery stages. A third capture shows the expanded test matrix. They accompany the report separately to preserve legibility. The unauthenticated browser displays stage statuses but requires sign-in to expand logs; the authenticated GitHub API supplied the archived text logs. Documentation added after the tested revision is identified separately in the submission manifest.

4. Difficulties, conclusions, and scope

The local Docker daemon was unavailable; Docker Desktop returned `kLSNoExecutableErr`. The hosted runner provided actual Docker evidence; local arm64 execution remains unverified. Archive inspection also found that Python's ZIP command flattened the workflow path. Including the complete `.github` directory fixed it; the final downloaded source was compared with the tested commit. Cached Tectonic resources resolved a compiler network error. All PDF pages were rendered and visually checked.

The result is a working CI and continuous-delivery path for the existing research tool. Artifacts expire after 30 days on GitHub, so downloaded evidence and the image are preserved in the submission archive. Automatic deployment to a separate test server or Kubernetes was not implemented, and no deployment bonus is claimed. AS6 remains outside this submission.